Practical: Breadth First Search & Iterative Depth First Search

1. Aim

- Implement the Breadth First Search algorithm to solve a given problem.
- Implement the Iterative Depth First Search algorithm to solve the same problem.
- Compare the performance and efficiency of both algorithms.

2. Python Code

```
from collections import deque
import time

# Graph representation
graph = {
   'A': ['B', 'C'],
   'B': ['D', 'E'],
   'C': ['F', 'G'],
   'D': [],
   'E': [],
   'F': [],
   'G': []
}

# Breadth First Search
def bfs(start, goal):
   queue = deque([start])
   visited = []

   while queue:
      node = queue.popleft()

      if node not in visited:
         visited.append(node)

         if node == goal:
            return visited

         queue.extend(graph[node])

   return visited


# Iterative Depth First Search
def ids(start, goal):
   depth = 0

   while True:
```

Explanation in simple language

Graph: The graph contains nodes A to G. A is the starting node, and G is the goal node.

Breadth First Search (BFS)

- queue = deque([start]) creates a queue containing the starting node.
- popleft() removes the first node from the queue.
- visited stores the nodes explored.
- queue.extend(graph[node]) adds the node's children to the queue.
- BFS explores nodes level by level.

Iterative Depth First Search (IDS)

- depth = 0 starts the search with a depth limit of zero.
- dls() searches only up to the given depth.
- stack.pop() removes the last node from the stack.
- If the goal is not found, the depth limit increases.
- The process repeats until the goal is found.

Performance comparison

- time.time() records the time before and after each search.
- The difference gives the approximate execution time.
- Both algorithms search for the same goal, G.

```
        result = dls(start, goal, depth)

        if result:
            return result

        depth += 1


# Depth Limited Search
def dls(start, goal, limit):
    stack = [(start, 0)]
    visited = []

    while stack:
        node, depth = stack.pop()

        if node not in visited:
            visited.append(node)

            if node == goal:
                return visited

            if depth < limit:
                for child in reversed(graph[node]):
                    stack.append((child, depth + 1))

    return None


# Compare performance
start = 'A'
goal = 'G'

t1 = time.time()
bfs_result = bfs(start, goal)
bfs_time = time.time() - t1

t1 = time.time()
ids_result = ids(start, goal)
ids_time = time.time() - t1

print("BFS Visited:", bfs_result)
print("BFS Time:", bfs_time)

print("IDS Visited:", ids_result)
print("IDS Time:", ids_time)
```

Practical 2: A* Search and Recursive Best-First Search
1. Aim: A SEARCH AND RECURSIVE BEST-FIRST SEARCH*
2. Easy Python code

```
import heapq

# Graph
graph = {
   'A': [('B', 1), ('C', 3)],
   'B': [('D', 3), ('E', 6)],
   'C': [('F', 5)],
   'D': [('G', 4)],
   'E': [('G', 2)],
   'F': [('G', 2)],
   'G': []
}

# Heuristic values
h = {'A': 7, 'B': 6, 'C': 4,
    'D': 4, 'E': 2, 'F': 1, 'G': 0}

# A* Search
def astar(start, goal):
   pq = [(h[start], 0, start, [start])]

   while pq:
      f, g, node, path = heapq.heappop(pq)

      if node == goal:
         return path, g

      for nxt, cost in graph[node]:
         heapq.heappush(
            pq, (g + cost + h[nxt],
                 g + cost, nxt, path + [nxt])
         )

# RBFS simplified for this practical
def rbfs(node, goal, cost, path):
   if node == goal:
      return path, cost

   for nxt, c in sorted(
      graph[node], key=lambda x: x[1] + h[x[0]]
   ):
      result = rbfs(nxt, goal, cost + c, path + [nxt])
      if result:
         return result

# Run both algorithms
print("A*:", astar('A', 'G'))
print("RBFS:", rbfs('A', 'G', 0, ['A']))
```

Explanation in simple language

A Search*

- graph stores the connected nodes and their costs.
- h stores the estimated cost from each node to the goal.
- heapq selects the node with the lowest priority value.
- g represents the actual cost travelled.
- f = g + h represents the estimated total cost.
- The algorithm returns the path when it reaches G.

Practical 3: Decision Tree Learning

1. Aim

DECISION TREE LEARNING

2. Easy Python code

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2, random_state=42
)

# Create decision tree
model = DecisionTreeClassifier(max_depth=3, random_state=42)

# Train model
model.fit(X_train, y_train)

# Predict output
y_pred = model.predict(X_test)

# Display accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Explanation in simple language

- load_iris() loads the Iris flower dataset.
- X stores flower measurements, and y stores flower categories.
- train_test_split() divides the data into training and testing sets.
- DecisionTreeClassifier() creates a decision tree model.
- max_depth=3 limits the tree to a maximum depth of 3.
- model.fit() trains the model.
- model.predict() predicts the flower categories.
- accuracy_score() calculates the accuracy.

Practical 4: Feed Forward Backpropagation Neural Network

1. Aim: FEED FORWARD BACKPROPAGATION NEURAL NETWORK

2. Easy Python code

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2, random_state=42
)

# Create neural network
model = MLPClassifier(
   hidden_layer_sizes=(10,),
   max_iter=1000,
   random_state=42
)

# Train model
model.fit(X_train, y_train)

# Predict output
y_pred = model.predict(X_test)

# Display accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Explanation in simple language

- load_iris() loads the Iris flower dataset.
- X stores the flower measurements.
- y stores the flower categories.
- train_test_split() divides the data into training and testing sets.
- MLPClassifier() creates a feed-forward neural network.
- hidden_layer_sizes=(10,) specifies one hidden layer with 10 neurons.
- model.fit() trains the network using backpropagation.
- model.predict() predicts the flower categories.
- accuracy_score() calculates the prediction accuracy.

Practical 5 — SVM

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2, random_state=42
)

# Create SVM model
model = SVC()

# Train model
model.fit(X_train, y_train)

# Predict output
y_pred = model.predict(X_test)

# Display accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Remember these six steps:

1. Import libraries.
2. Load the dataset.
3. Split the data.
4. Create the model.
5. Train and predict.
6. Print accuracy.

Expected output: an accuracy score between 0 and 1; the exact value can vary with the dataset and settings.

Practical 6: AdaBoost Ensemble Learning

1. Aim

ADABOOST ENSEMBLE LEARNING

2. Easy Python code

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import accuracy_score

# Load dataset
data = load_breast_cancer()
X = data.data
y = data.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2, random_state=42
)

# Create AdaBoost model
model = AdaBoostClassifier(n_estimators=10, random_state=42)

# Train model
model.fit(X_train, y_train)

# Predict output
y_pred = model.predict(X_test)

# Display accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Explanation in simple language

- load_breast_cancer() loads a sample dataset for classifying tumours as benign or malignant.
- X stores the input features.
- y stores the target classes.
- train_test_split() divides the data into training and testing sets.
- AdaBoostClassifier() creates the AdaBoost model.
- n_estimators=10 sets the number of weak learners to 10.
- model.fit() trains the model.
- model.predict() predicts the classes.
- accuracy_score() calculates the model's accuracy.

Practical 7: Naive Bayes Classification — Easy Version

Aim: To implement a Naive Bayes classification algorithm using Python.

1. Simple Python code

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

# Load dataset
data = load_iris()
X = data.data
y = data.target

# Split the data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create and train the model
model = GaussianNB()
model.fit(X_train, y_train)

# Make predictions
y_pred = model.predict(X_test)

# Display accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Simple explanation

- Naive Bayes: A classification algorithm based on Bayes' theorem.
- Dataset: The Iris dataset contains flower measurements and species labels.
- Train-test split: Divides data into training and testing sets.
- GaussianNB(): Creates a Naive Bayes model for numerical data.
- fit(): Trains the model.
- predict(): Predicts the flower classes.
- accuracy_score(): Calculates the percentage of correct predictions.

Practical 8: K-Nearest Neighbors (K-NN) Classification

1. Aim

K-NEAREST NEIGHBORS (K-NN) CLASSIFICATION

2. Easy Python code

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2, random_state=42
)

# Create KNN model
model = KNeighborsClassifier(n_neighbors=5)

# Train model
model.fit(X_train, y_train)

# Predict output
y_pred = model.predict(X_test)

# Display accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Explanation in simple language

- load_iris() loads the Iris flower dataset.
- X stores the flower measurements.
- y stores the flower categories.
- train_test_split() divides the data into training and testing sets.
- KNeighborsClassifier(n_neighbors=5) creates a K-NN model using five nearest neighbors.
- model.fit() prepares the model using training data.
- model.predict() predicts the categories of test flowers.
- accuracy_score() calculates the prediction accuracy.

Practical 9: Association Rule Mining — Easy Version

Aim: To implement Association Rule Mining using Python to discover relationships between items.

1. Simple Python code

Install the required library:

```
pip install mlxtend pandas
```

```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

# Sample transactions
transactions = [
   ['Milk', 'Bread'],
   ['Milk', 'Bread', 'Butter'],
   ['Bread', 'Butter'],
   ['Milk', 'Bread'],
   ['Milk', 'Butter']
]

# Convert transactions into a table
encoder = TransactionEncoder()
data = encoder.fit(transactions).transform(transactions)
df = pd.DataFrame(data, columns=encoder.columns_)

# Find frequent itemsets
items = apriori(df, min_support=0.4, use_colnames=True)

# Generate association rules
rules = association_rules(
   items, metric="confidence", min_threshold=0.5
)

print("Frequent Itemsets:")
print(items)

print("\nAssociation Rules:")
print(rules[['antecedents', 'consequents', 'support', 'confidence']])
```

Simple explanation

- Association Rule Mining: Finds relationships between items frequently purchased together.
- Transaction: A collection of items purchased in one shopping visit.
- Apriori: Finds frequent combinations of items.
- Support: How often an item or combination appears in all transactions.
- Confidence: How often the consequent occurs when the antecedent occurs.
- Association rule: A relationship written as Milk → Bread.

Practical 10: Demo of TensorFlow Tools — Easy Version

Aim: To demonstrate the basic use of TensorFlow tools for a simple AI application.

1. Simple Python code

Install TensorFlow first if it is not installed:

```
pip install tensorflow
import tensorflow as tf

# Create sample data
x = tf.constant([1, 2, 3, 4, 5], dtype=tf.float32)
y = tf.constant([2, 4, 6, 8, 10], dtype=tf.float32)

# Create a simple AI model
model = tf.keras.Sequential([
   tf.keras.layers.Dense(1, input_shape=(1,))
])

# Compile the model
model.compile(optimizer="sgd", loss="mse")

# Train the model
model.fit(x, y, epochs=100, verbose=0)

# Predict a new value
result = model.predict(tf.constant([[6.0]]), verbose=0)

print("Prediction:", result[0][0])
```

Simple explanation

- TensorFlow: A library used to build and train AI models.
- Data: The program uses numbers where the output is twice the input.
- Model: A neural network with one output neuron.
- Compile: Configures how the model learns.
- Train: The model learns from the given examples.
- Predict: The model estimates the output for input 6.

Practical 1: Breadth First Search (BFS) and Iterative Depth First Search (DFS)

Q1. What is BFS?
BFS is a search algorithm that visits nodes level by level.

Q2. What is DFS?
DFS explores one branch as deeply as possible before backtracking.

Q3. Which data structure is used in BFS?
A queue (FIFO).

Q4. Which data structure is used in DFS?
A stack (LIFO), or recursion.

Q5. What is the difference between BFS and DFS?
BFS explores level by level and usually requires more memory. DFS explores deeply and usually uses less memory.

Q6. What is a graph?
A graph is a collection of nodes (vertices) connected by edges.

Q7. What is a visited set?
It stores nodes already visited to avoid processing them repeatedly.

Q8. What is the advantage of BFS?
It finds the shortest path in an unweighted graph.

Q9. What is the advantage of DFS?
It usually requires less memory than BFS.

Q10. What is iterative DFS?
It implements DFS using a stack and a loop instead of recursion.

Practical 2: A* Search and Recursive Best-First Search (RBFS)

Q1. What is A search?*
A* is an informed search algorithm that uses path cost and a heuristic to find a path to the goal.

Q2. What is the A evaluation function?*

$$f(n) = g(n) + h(n)$$

Q3. What is \(g(n)\)?
The actual cost from the starting node to the current node.

Q4. What is \(h(n)\)?
The estimated cost from the current node to the goal.

Q5. What is a heuristic?
A heuristic is an estimate used to guide a search toward the goal.

Q6. What is RBFS?
Recursive Best-First Search is a heuristic search algorithm that uses recursion and limits how much search information it keeps in memory.

Q7. What is the main difference between A and RBFS?*
A* generally stores many frontier nodes, while RBFS uses recursion and backtracking to reduce memory usage.

Q8. What is a goal node?
The node representing the desired destination or solution.

Q9. What is an informed search algorithm?
A search algorithm that uses additional knowledge, such as a heuristic, to guide its search.

Q10. Where is A used?*
It is used in route finding, maps, robotics, and pathfinding in games.

Practical 3: Decision Tree Learning

Q1. What is a decision tree?
A decision tree is a machine-learning model that makes decisions using feature-based rules.

Q2. What is a root node?
The topmost node of a decision tree.

Q3. What is a leaf node?
A terminal node that provides the final prediction.

Q4. What is classification?
Classification predicts a category or class.

Q5. What is DecisionTreeClassifier()?
It creates a decision tree classification model in scikit-learn.

Q6. What is fit()?
It trains the model using training data.

Q7. What is predict()?
It predicts the class of new or test data.

Q8. What is overfitting?
Overfitting occurs when a model learns the training data too closely and performs poorly on unseen data.

Q9. Why use max_depth?
It limits the depth of the tree and can help reduce overfitting.

Q10. Give one application of a decision tree.
Loan approval, medical diagnosis, or customer classification.

Practical 4: Feed Forward Backpropagation Neural Network

Q1. What is an artificial neural network (ANN)?
An ANN is a machine-learning model made of interconnected artificial neurons.

Q2. What is a feed-forward neural network?
A network in which information moves from the input layer toward the output layer.

Q3. What is backpropagation?
It adjusts network weights by propagating prediction errors backward through the network.

Q4. What is a neuron?
A basic processing unit of a neural network.

Q5. What is a hidden layer?
A layer between the input and output layers that learns patterns from data.

Q6. What is an activation function?
A function that helps a neural network learn nonlinear patterns.

Q7. What is MLPClassifier()?
A scikit-learn class used to build a multilayer perceptron for classification.

Q8. What is an epoch?
One complete pass through the training dataset.

Q9. What is a loss function?
It measures the difference between actual and predicted outputs.

Q10. Where are neural networks used?
Image recognition, speech recognition, prediction, and natural-language processing.

Practical 5: Support Vector Machine (SVM)

Q1. What is SVM?
Support Vector Machine is a supervised machine-learning algorithm used mainly for classification and also for regression.

Q2. What is a hyperplane?
A decision boundary that separates classes in a feature space.

Q3. What are support vectors?
The data points closest to the decision boundary that help determine its position.

Q4. What is the margin in SVM?
The distance between the decision boundary and the nearest data points from the classes.

Q5. What is the objective of SVM?
To find a decision boundary with the largest possible margin, subject to the model's constraints.

Q6. What is a kernel?
A function that helps SVM model nonlinear decision boundaries.

Q7. What is SVC()?
It creates a Support Vector Classification model in scikit-learn.

Q8. What is the Iris dataset?
A dataset containing measurements of three iris flower species.

Q9. Why split data into training and testing sets?
To train the model on one part and evaluate it on unseen data.

Q10. Give applications of SVM.
Text classification, image recognition, and spam detection.

Practical 6: AdaBoost Ensemble Learning

Q1. What is ensemble learning?
It combines multiple models to produce a prediction.

Q2. What is AdaBoost?
AdaBoost stands for Adaptive Boosting. It combines weak learners into a stronger model.

Q3. What is a weak learner?
A model that performs only slightly better than random guessing on a suitable task.

Q4. How does AdaBoost work?
It trains learners sequentially, giving more attention to examples that earlier learners classified incorrectly.

Q5. What is boosting?
A technique that builds models sequentially to improve predictive performance.

Q6. What is AdaBoostClassifier()?
A scikit-learn class that implements AdaBoost for classification.

Q7. What is n_estimators?
It specifies the number of boosting estimators to use.

Q8. What is accuracy?
The proportion of predictions that are correct.

Q9. What is one advantage of AdaBoost?
It can combine simple learners into a more accurate model.

Q10. Give one application of AdaBoost.
Fraud detection, medical classification, or face detection.

Practical 7: Naive Bayes Classification

Q1. What is Naive Bayes?
It is a classification algorithm based on Bayes' theorem.

Q2. What is Bayes' theorem?
It calculates the probability of an event using prior information and evidence.

Q3. Why is it called “naive”?
It assumes that features are conditionally independent given the class.

Q4. What is Gaussian Naive Bayes?
It is a Naive Bayes algorithm that assumes numerical features follow a Gaussian distribution within each class.

Q5. What is classification?
Classification assigns data to predefined categories.

Q6. What is the Iris dataset?
A dataset containing measurements of three iris flower species.

Q7. What is GaussianNB()?
It creates a Gaussian Naive Bayes classification model in scikit-learn.

Q8. What is fit()?
It trains the model using the provided data.

Q9. What does accuracy mean?
Accuracy is the proportion of correct predictions among all predictions.

Q10. Where is Naive Bayes used?
Spam filtering, sentiment analysis, and document classification.

Practical 8: K-Nearest Neighbors (K-NN) Classification

Q1. What is K-NN?
K-Nearest Neighbors is a supervised algorithm that predicts a class using the nearest training examples.

Q2. What does K represent?
The number of nearest neighbors considered for prediction.

Q3. What is KNeighborsClassifier()?
It creates a K-NN classification model in scikit-learn.

Q4. How does K-NN work?
It finds the nearest training points and predicts the class based on their labels, usually by majority vote.

Q5. What happens if K is too small?
The model may be sensitive to noise and individual data points.

Q6. What happens if K is too large?
The model may overlook local patterns and produce less suitable predictions.

Q7. Why is feature scaling important in K-NN?
Because K-NN uses distances, features with larger numerical scales can otherwise dominate.

Q8. Is K-NN supervised or unsupervised?
It is a supervised learning algorithm.

Q9. What is Euclidean distance?
A measure of straight-line distance between points.

Q10. Give applications of K-NN.
Image classification, recommendation, and pattern recognition.

Practical 9: Association Rule Mining

Q1. What is Association Rule Mining?
It discovers relationships between items that frequently occur together.

Q2. What is the Apriori algorithm?
It finds frequent itemsets and helps generate association rules.

Q3. What is a transaction?
A collection of items recorded together, such as products in one shopping basket.

Q4. What is a frequent itemset?
A group of items that appears in enough transactions to meet the minimum support threshold.

Q5. What is support?
Support is the proportion of all transactions containing an itemset.

Q6. What is confidence?
Confidence measures how often the consequent appears in transactions containing the antecedent.

Q7. What does min_support=0.4 mean?
The itemset must appear in at least 40% of the transactions.

Q8. What does min_threshold mean in the association rules code?
It sets the minimum confidence required for a rule when the selected metric is confidence.

Q9. What does a rule like Milk → Bread mean?
It indicates that transactions containing Milk are associated with Bread.

Q10. Give a real-world application.
Supermarkets use association rules to identify products that customers frequently purchase together.

Practical 10: Demo of TensorFlow Tools and AI Chatbot

Q1. What is Artificial Intelligence (AI)?
AI is the ability of machines to perform tasks that normally require human intelligence.

Q2. What is TensorFlow?
TensorFlow is an open-source library used to develop and train machine-learning models.

Q3. What is a chatbot?
A chatbot is a software application that interacts with users through text or speech.

Q4. What is ChatGPT?
ChatGPT is an AI assistant that understands prompts and generates responses.

Q5. What is a neural network?
A neural network is a machine-learning model made of interconnected artificial neurons.

Q6. What is tf.keras.Sequential()?
It creates a neural network by arranging layers in sequence.

Q7. What is Dense(1)?
It creates a fully connected layer with one output neuron.

Q8. What is an optimizer?
An optimizer updates model parameters to reduce prediction error during training.

Q9. What is a loss function?
A loss function measures the difference between the model's predictions and the target values.

Q10. What is an epoch?
One complete pass through the training dataset.

Q11. What does model.fit() do?
It trains the model using the provided data.

Q12. What does model.predict() do?
It uses the trained model to generate predictions for new inputs.

Q13. What is the purpose of the TensorFlow example with 1 → 2, 2 → 4, and so on?
It teaches the model a numerical pattern in which the output is approximately twice the input.

Q14. What is the expected prediction for input 6?
Approximately 12.

Q15. Give applications of AI.
Healthcare, education, banking, transportation, customer support, and business automation.